CrossVul-DG: Class-Conditional Domain Generalization for Cross-Project Software Vulnerability Detection

Anonymous Author(s)
Affiliation withheld for double-blind review
Email: anonymous@example.com

***Abstract*—Deep-learning models for software vulnerability detection (SVD) degrade sharply on projects unseen during training, because of domain shift in coding conventions, naming, and architecture. Domain Adaptation (DA) needs target-project data at training time, which is unrealistic for proactive deployment; recent Domain Generalization (DG) for SVD takes the *project* as the only unit of invariance and is *class-blind*, ignoring the severe class imbalance of real vulnerability data. We argue cross-project SVD is a *coupled* domain-shift and class-imbalance problem, and we propose CC-DRO (Class-Conditional Distributionally-Robust Optimization): it forms distributional groups over *(project* $\times$ *class)* cells and adds a cross-project *positive-class* risk-invariance penalty, so a model's vulnerability sensitivity is robust *and* consistent across unseen projects. We study CC-DRO against ERM, IRM, GroupDRO, two DA baselines (which access unlabelled target data), and a pseudo-domain variant, under a leave-one-project-out (LOPO) protocol on MegaVul with a chronological 8:1:1 split, a shared imbalanced-data protocol (class-weighted loss, validation-tuned threshold, PR-AUC as the primary metric), and three encoder backbones. We expect CC-DRO to improve cross-project PR-AUC and, in particular, to narrow the cross-project *vulnerability-recall gap* relative to class-blind DG, with only a marginal seen-project regression. The figures in this manuscript are illustrative placeholders pending the full run; we release code and pre-processed splits for reproducibility.**

***Keywords*—software vulnerability detection, domain generalization, class imbalance, distributionally robust optimization, IRM, MegaVul, cross-project, CodeBERT**

I. Introduction

Software vulnerabilities cause large-scale economic and societal harm, and automated detection is an intensive research focus, with deep-learning detectors achieving strong in-distribution F1 [2]–[5]. Yet when these models are trained on one codebase and applied to another, performance often drops substantially [6], [7]. This cross-project gap is a fundamental barrier to deployment: a security team rarely has labelled vulnerabilities in the target codebase before scanning it.

Placeholder notice. This is a study-protocol draft. Every quantitative value in the abstract and in Section V is an illustrative placeholder produced only to exercise the released analysis pipeline; the numbers are *not* measured results on MegaVul. They must be replaced with values from the experiments in Section IV before this manuscript is submitted.

A natural response is Domain Adaptation (DA), aligning the model using unlabelled or partially labelled target data [8], [9]. DA presupposes target data at training time, which is unrealistic for proactive deployment (e.g., a vendor shipping a detector to unknown customer codebases). Domain Generalization (DG) instead learns representations that generalise to *any* unseen domain using only source data at training time [10]–[12]. DG is mature in computer vision; in SVD it has begun to appear only recently, e.g. the cluster-based ensemble method CEVD [1].

The coupled problem. Cross-project SVD is *not* a pure domain-shift problem. Real vulnerability datasets are highly imbalanced — in MegaVul roughly one function in twenty is vulnerable. Off-the-shelf DG objectives (IRM, GroupDRO) take the project as the unit of invariance and are *class-blind*: under $\sim$1:19 imbalance, the worst-case *per-project* risk can be minimised by excelling on the dominant safe class, so the rare-but-critical vulnerability signal is under-served, and nothing in the objective makes vulnerability *recall* consistent across projects. A detector can thus look strong on aggregate while its ability to flag real vulnerabilities swings widely from project to project.

This paper. We present CrossVul-DG, an empirical study of invariance/robustness-based DG for cross-project SVD on MegaVul [13], and we introduce **CC-DRO**, a class-conditional robust objective for the coupled problem. CC-DRO (i) replaces project-level groups with (project $\times$ class) cells so the worst case explicitly includes the *(project, vulnerable)* cells, and (ii) adds a penalty equal to the variance, across projects, of the positive-class risk, forcing vulnerability sensitivity to be invariant across projects. All methods share a single imbalanced-data protocol so the comparison is fair, and we test generality across three code encoders (CodeBERT [14], GraphCodeBERT [15], UniXcoder [16]).

Research questions.

- **RQ1.** Does DG (IRM, GroupDRO, or CC-DRO) improve cross-project PR-AUC over standard ERM fine-tuning?
- **RQ2.** How does DG compare with DA methods that have access to (unlabelled) target data?
- **RQ3.** *(Core)* Does class-conditional invariance (CC-DRO) reduce the cross-project *vulnerability-recall gap* relative to class-blind DG?
- **RQ4.** What is the trade-off between cross-project gain and seen-project performance?
- **RQ5.** Does the CC-DRO gain transfer across encoder backbones?

- **RQ6.** *(Optional)* How do prompted code-LLMs perform as zero-/few-shot cross-project detectors?

Contributions.

1) We frame cross-project SVD as a *coupled* domain-shift and class-imbalance problem and propose **CC-DRO**, to our knowledge the first class-conditional robust DG objective for SVD, with an explicit cross-project positive-class invariance penalty.
2) We provide a reproducible LOPO protocol on MegaVul with a shared imbalanced-data treatment (class-weighted loss, validation-tuned decision threshold, PR-AUC primary metric, and a leakage-free DA target split), and statistical significance testing.
3) We adapt IRM, GroupDRO and a pseudo-domain extension to the same protocol as baselines, and evaluate across three code encoders.
4) We release pre-processed splits, training pipelines, and evaluation scripts.[1] (Quantitative gains in this draft are placeholders to be confirmed by the released pipeline.)

II. Background and Related Work

A. Deep Learning for Vulnerability Detection

Early sequence detectors such as VulDeePecker [2] used LSTMs over token sequences. Graph methods, notably Devign [3] and the ReVeal study [6], encoded code property graphs with GNNs. Transformer encoders — CodeBERT [14], GraphCodeBERT [15], UniXcoder [16] — are now the de facto backbone, with LineVul [4] a strong line-level baseline and VulBERTa [5] a simplified recipe. LLM studies [17] report that prompted detectors remain expensive and unstable across projects.

B. Cross-Project Vulnerability Detection

Chakraborty et al. [6] documented the severity of cross-project drops; Steenhoek et al. [7] confirmed them under a controlled study. Two recent lines target generalization directly. CEVD [1] is a DG method that clusters code embeddings into pseudo-domains and trains an ensemble of per-domain experts fused by a domain classifier, with no target data. VulGate [18] is an orthogonal, data-centric effort: a large de-duplicated dataset (236,663 functions, with a 500-sample expert-verified cross-project test set) showing data quality and model choice strongly affect generalization. CrossVul-DG differs from CEVD by applying explicit invariance/robustness objectives over the natural project environments and, with CC-DRO, by conditioning the robust objective on *class*; it is complementary to VulGate, which improves the data rather than the objective.

C. Domain Generalization and Class Imbalance

DG learns predictors that generalise to unseen domains [12]. IRM [10] penalises representations on which the optimal classifier differs across environments; GroupDRO [11] minimises the worst-case group risk; gradient-matching (Fish [19]) and Mixup-style augmentations extend the family. Surveys [20], [21] note DG's maturity in vision/NLP but limited adoption in software engineering. Crucially, all of these define groups by domain *only*. CC-DRO defines groups by domain $\times$ class and adds a positive-class invariance term — a class-conditional robust objective tailored to imbalanced SVD.

[1] https://github.com/anonymous/crossvul-dg (anonymised for review).

III. The CrossVul-DG Framework

A. Problem Formulation

Let $\mathcal{P} = \{p_1, \ldots, p_K\}$ be the source projects. Each p_k provides $D_k = \{(x_i^{(k)}, y_i^{(k)})\}$ with function x_i and label $y_i \in \{0, 1\}$ (vulnerable or not). The positive rate $\pi_k = \Pr[y = 1 \mid p_k]$ is small (≈ 0.05). We seek $f_\theta : \mathcal{X} \to \{0, 1\}$ that generalises to an unseen project $p_* \notin \mathcal{P}$ using only $\bigcup_k D_k$ at training time (no target access), and whose *positive-class* performance is consistent across projects.

B. Backbone Encoder

We use a fine-tuned code encoder g_ϕ (CodeBERT by default; GraphCodeBERT and UniXcoder for generality) producing a 768-d first-token representation, followed by a two-layer MLP h_ψ with a single logit output. We write $f_\theta = h_\psi \circ g_\phi$, $\theta = (\phi, \psi)$.

C. Imbalanced-Setting Protocol (shared by all methods)

Because we keep MegaVul's natural ≈1:19 ratio, every method shares the following, so differences reflect the objective, not the imbalance treatment:

- **Class-weighted loss.** Per-sample BCE with $\text{pos_weight} = N_{\text{neg}}/N_{\text{pos}}$ on the training fold (capped), so optimisation does not collapse to the safe class.
- **Model selection on validation PR-AUC** (threshold-free; validation-F1 at a fixed 0.5 threshold is unreliable under imbalance).
- **Validation-tuned decision threshold** (max-F1), applied unchanged at test — no test-set peeking.
- **Primary metric PR-AUC**, with F1 at the tuned threshold, recall at precision ≥ 0.5 (R@P50), and *worst-project F1*.
- **Leakage-free DA target.** DA baselines use the chronologically *earlier* portion of the held-out project as the unlabelled target; *all* methods are scored on the *later* portion (identical test set).

D. ERM, IRM, GroupDRO (baselines)

Let $R_k(\theta) = \mathbb{E}_{D_k}[\, w_y\, \ell(f_\theta(x), y)\,]$ be the class-weighted risk on project k (w_y the pos_weight on positives). ERM minimises $\frac{1}{K}\sum_k R_k$. IRM [10] adds

$$\mathcal{L}_{\text{IRM}} = \sum_{k=1}^{K} R_k(\theta) + \lambda \sum_{k=1}^{K} \left\| \nabla_{w \mid w=1} R_k(w \cdot f_\theta) \right\|^2, \quad (1)$$

ramping λ after a warm-up. GroupDRO [11] minimises the worst-case project risk $\min_\theta \max_{q \in \Delta_K} \sum_k q_k R_k(\theta)$ via exponentiated-gradient updates $q_k \leftarrow q_k \exp(\eta_q R_k)$; $q \leftarrow q / \sum_{k'} q_{k'}$.

Algorithm 1 CC-DRO optimisation step

Require: per-project batches $\{B_k\}_{k=1}^{K}$; cell weights q; pos_weight w; penalty weight λ_{inv}

$\mathcal{C} \leftarrow \{\}$ ▷ cell → mean loss; $\mathcal{R}^+ \leftarrow [\,]$
for each project k with batch B_k **do**
  $\ell_i \leftarrow w^{\,y_i}\,\mathrm{BCE}(f_\theta(x_i), y_i)$ for $(x_i, y_i) \in B_k$
  for $c \in \{0, 1\}$ present in B_k **do**
    $\mathcal{C}[(k, c)] \leftarrow \mathrm{mean}\{\ell_i : y_i = c\}$
  end for
  if $(k, 1) \in \mathcal{C}$ **then**
    append $\mathcal{C}[(k, 1)]$ to $\mathcal{R}^+$
  end if
end for
$q_g \leftarrow q_g \exp(\eta_q\, \mathcal{C}[g])$ for $g \in \mathcal{C}$; $q \leftarrow q / \sum q$ ▷ DRO update
$L \leftarrow \sum_{g \in \mathcal{C}} q_g\, \mathcal{C}[g]$
if $|\mathcal{R}^+| \geq 2$ **then**
  $L \leftarrow L + \lambda_{\text{inv}}\, \mathrm{Var}(\mathcal{R}^+)$
end if
back-propagate L; update θ

E. CC-DRO: Class-Conditional DRO (proposed)

CC-DRO changes the group structure and adds an invariance term. **(1) Class-conditional groups.** Groups are cells $g = (p, c)$ for project p and class $c \in \{0, 1\}$, giving $2K$ cells; GroupDRO is run over the cells, so the worst case explicitly includes the $(p, 1)$ *vulnerable* cells. **(2) Positive-class invariance.** With $R_{p,\text{pos}}$ the mean weighted loss on project p's positive samples, we penalise their variance across projects. The per-step objective is

$$\mathcal{L}_{\text{CC-DRO}} = \sum_g q_g R_g(\theta) \;+\; \lambda_{\text{inv}}\; \mathrm{Var}_p\big(R_{p,\text{pos}}\big), \qquad (2)$$

where g ranges over the cells present in the step and q are exp-gradient weights over cells. Minimising the worst-case cell risk concentrates capacity on the hardest *(project, vulnerable)* cells; minimising the positive-risk variance makes vulnerability recall consistent across projects. Cells absent from a mini-batch are omitted that step (rare-positive projects may yield no positives in a small batch); the penalty is applied only when ≥ 2 projects contribute positives, and λ_{inv} is ramped after a warm-up. Algorithm 1 gives one optimisation step.

F. Pseudo-Domain Clustering (long-tail extension)

MegaVul has a long tail: the top-10 projects cover most samples while ∼980 are small. We optionally cluster small projects in embedding space (mean function embedding per project) into C pseudo-domains with k-means and add them as extra environments for GroupDRO (PD+DRO). Clustering code embeddings to expose latent domains is also central to CEVD [1]; here it is an auxiliary source of environments for a robust objective rather than the basis for an ensemble. Algorithm 2 summarises the procedure.

IV. Experimental Setup

A. Dataset

We use MegaVul [13], a large real-world C/C++ dataset: 17,380 vulnerabilities across 169 CWE types (2006–2023) from 992 repositories; the function-level release has ≈17,000 vulnerable and ≈320,000 non-vulnerable functions with CVE identifiers and commit timestamps. We select the top-10 projects by function count as LOPO environments (Table I); the exact post-deduplication share is produced by the preprocessing script.

Algorithm 2 Pseudo-domain construction (PD+DRO)

Require: top-K projects $\mathcal{P}_{\text{top}}$; tail projects $\mathcal{P}_{\text{tail}}$; cluster count C

compute mean embedding $\bar{z}_p$ for each $p \in \mathcal{P}_{\text{tail}}$ via the encoder
cluster $\{\bar{z}_p\}$ into C groups with k-means
environments $\mathcal{E} \leftarrow \mathcal{P}_{\text{top}} \cup \{c_1, \ldots, c_C\}$
train f_θ by minimising GroupDRO over $\mathcal{E}$
return f_θ

TABLE I. Top-10 MegaVul projects used as LOPO environments. *Placeholder: exact post-deduplication counts are produced by* `prepare_data.py` *and will replace the dashes.*

Project	#Func.	#Vuln.	Vuln. %
Linux	—	—	—
Chromium	—	—	—
FFmpeg	—	—	—
QEMU	—	—	—
PHP-src	—	—	—
ImageMagick	—	—	—
OpenSSL	—	—	—
TensorFlow	—	—	—
Wireshark	—	—	—
systemd	—	—	—

B. Splits and Protocol

A chronological 8:1:1 train/val/test split by commit date over the pooled selected samples (cut at the 0.8 and 0.9 time quantiles), rather than by year. For cross-project evaluation we use LOPO: each top-10 project is held out in turn; the held-out project is split chronologically into an earlier DA-target portion and a later test portion (Section III-C). We report mean ± std across seeds (three for the main table; two for the backbone study) and test significance with a paired Wilcoxon signed-rank test over the 10 folds.

C. Baselines and Methods

ERM-FT (fine-tuning, ignores project labels); **DA-DANN** [8] and **DA-CORAL** [9] (use the earlier portion of the held-out project as unlabelled target — a reference, not strict DG); **CrossVul-DG (IRM)**, **(DRO)**; **CC-DRO** (proposed); and **(PD+DRO)** (pseudo-domain + GroupDRO).

D. Implementation Details

Encoder `microsoft/codebert-base` (and `unixcoder-base`, `graphcodebert-base` for Tier 2); sequence length 512; batch size 32 (per-environment for DG); AdamW, lr 2×10^{-5}, weight decay 10^{-2}; 10 epochs, early stopping on validation PR-AUC; IRM $\lambda_{\max}$=10, warm-up 1 epoch; GroupDRO/CC-DRO η_q=0.01; CC-DRO λ_{inv}=1 (ablated in $\{0, 0.1, 1, 10\}$), warm-up 1 epoch; pseudo-domain C=5. All runs fit a single 24 GB GPU.

TABLE II. LEAVE-ONE-PROJECT-OUT RESULTS ON MEGAVUL (CODEBERT), MEAN OVER 10 FOLDS (EACH AVERAGED OVER SEEDS). PR-AUC IS PRIMARY; WORST-F1 IS THE ROBUSTNESS NUMBER. † USES UNLABELLED TARGET DATA. *Illustrative placeholders — not measured.*

Method	PR-AUC	$F1_{thr}$	R@P50	worst-F1
ERM-FT	—	—	—	—
DA-CORAL†	—	—	—	—
DA-DANN†	—	—	—	—
IRM	—	—	—	—
GroupDRO	—	—	—	—
CC-DRO (ours)	—	—	—	—
PD+DRO	—	—	—	—

TABLE III. RQ3: CROSS-PROJECT VULNERABILITY-RECALL GAP (MAX−MIN PER-PROJECT RECALL AT THE TUNED THRESHOLD; LOWER IS BETTER) AND MEAN VULNERABILITY RECALL. *Placeholders — not measured.*

Method	Recall gap ↓	Mean recall ↑
IRM	—	—
GroupDRO	—	—
CC-DRO (ours)	—	—

E. Metrics

PR-AUC (primary), F1 at the validation-tuned threshold, recall at precision ≥ 0.5 (R@P50), ROC-AUC, and worst-project F1. Following [6] we also report the per-project F1 range.

V. EXPECTED RESULTS (ILLUSTRATIVE)

Placeholder notice. Every number in this section is a placeholder obtained by running the released pipeline on tiny synthetic data, included only to show the reporting format and verify the analysis scripts end-to-end. These are *not* measured results on MegaVul. Replace them with the output of `run_full_sweep.sh` before drawing any conclusion or submitting.

Tables II–V illustrate the intended presentation and the hypothesised direction of effects (CC-DRO $\geq$ class-blind DG $\geq$ DA $\geq$ ERM on cross-project PR-AUC, and a smaller vulnerability-recall gap for CC-DRO). They do not constitute evidence until the experiments are executed.

A. RQ1: DG vs. ERM

Table II shows the intended LOPO summary, with placeholders consistent with the hypothesis that robust DG (and CC-DRO in particular) improves cross-project PR-AUC over ERM.

B. RQ2: DG vs. DA

The key question is whether DG can match DA despite seeing no target labels. The placeholders encode the hypothesised outcome (DG $\geq$ DA); a plausible mechanism, if confirmed, is that with many natural source domains, invariance objectives extract more transferable signal than adversarial alignment to a single target.

C. RQ3: Vulnerability-recall gap (core)

Table III is the central evaluation: we expect CC-DRO to reduce the per-project recall gap relative to class-blind IRM and GroupDRO, by construction of its class-conditional groups and positive-class invariance penalty (Eq. 2).

TABLE IV. SEEN VS. UNSEEN PROJECT F1 (AVERAGED OVER THE 10 LOPO FOLDS). *Placeholders — not measured.*

Method	Seen F1	Unseen F1
ERM-FT	—	—
CC-DRO (ours)	—	—
Δ	—	—

TABLE V. RQ5: BACKBONE GENERALITY (PR-AUC, MEAN OVER 10 FOLDS). *Placeholders — not measured.*

Backbone	ERM-FT	GroupDRO	**CC-DRO**
CodeBERT	—	—	—
GraphCodeBERT	—	—	—
UniXcoder	—	—	—

D. RQ4: Seen vs. unseen trade-off

Table IV reports the hypothesised small seen-project regression in exchange for a larger unseen-project gain.

E. RQ5: Backbone generality

Table V tests whether the CC-DRO gain holds, in direction, across CodeBERT, GraphCodeBERT, and UniXcoder.

VI. DISCUSSION

Why might class-conditioning help? Under heavy imbalance, a domain-only robust objective can satisfy its worst-case constraint through the majority class. Conditioning groups on class forces the optimiser to confront the *(project, vulnerable)* cells directly, and the positive-risk variance term discourages solutions that detect vulnerabilities well on some projects and poorly on others. Whether this materialises on MegaVul is an empirical question.

Practical implications (if confirmed). Tools shipped to many customer codebases face exactly the DG setting. If the hypotheses hold, practitioners should (i) gather as many distinct source projects as possible, (ii) prefer class-conditional robust objectives under imbalance, and (iii) accept a small seen-project regression for substantial unseen-project gains in vulnerability recall.

Threats to validity. *Internal:* IRM/DRO are hyperparameter-sensitive; we report mean ± std and provide CC-DRO ablations. *External:* findings are bound to C/C++ MegaVul; cross-language generalisation is future work. *Construct:* MegaVul labels inherit upstream noise; the leakage-free DA split reduces the per-fold test size, a trade-off we report. *Conclusion:* until the protocol is run, the numbers are illustrative and support no conclusions.

VII. CONCLUSION

We recast cross-project SVD as a coupled domain-shift and class-imbalance problem and proposed **CC-DRO**, a class-conditional robust DG objective with a cross-project positive-class invariance penalty. Within a reproducible LOPO protocol on MegaVul — shared imbalanced-data treatment, PR-AUC primary metric, leakage-free DA comparison, and three backbones — the study will test whether class-conditioning narrows the cross-project vulnerability-recall gap left by class-blind DG, and whether DG can rival DA without target

Fig. 1 placeholder: ablation of CC-DRO $\lambda_{\text{inv}} \in \{0, 0.1, 1, 10\}$ (PR-AUC and recall gap) and pseudo-domain count $C \in \{0, 2, 5, 10, 20\}$.

Fig. 1. Planned ablations; curves are schematic until measured.

access. It complements cluster-based DG (CEVD [1]) and the data-centric VulGate effort [18]. Future work includes cross-language settings, causal feature discovery, and combining DG with retrieval-augmented LLM detectors.

Data and Code Availability

Pre-processed splits, training pipelines, and evaluation scripts are available at https://github.com/anonymous/crossvul-dg (anonymised for review).

References

[1] J. Liu, "CEVD: Cluster-based ensemble learning for cross-project vulnerability detection," *Future Internet*, vol. 18, no. 2, p. 85, 2026.

[2] Z. Li, D. Zou, S. Xu, X. Ou, H. Jin, S. Wang, Z. Deng, and Y. Zhong, "VulDeePecker: A deep learning-based system for vulnerability detection," in *Proc. NDSS*, 2018.

[3] Y. Zhou, S. Liu, J. Siow, X. Du, and Y. Liu, "Devign: Effective vulnerability identification by learning comprehensive program semantics via graph neural networks," in *NeurIPS*, 2019.

[4] M. Fu and C. Tantithamthavorn, "LineVul: A transformer-based line-level vulnerability prediction," in *Proc. MSR*, 2022, pp. 608–620.

[5] H. Hanif and S. Maffeis, "VulBERTa: Simplified source code pre-training for vulnerability detection," in *Proc. IJCNN*, 2022.

[6] S. Chakraborty, R. Krishna, Y. Ding, and B. Ray, "Deep learning based vulnerability detection: Are we there yet?" *IEEE Trans. Softw. Eng.*, vol. 48, no. 9, pp. 3280–3296, 2022.

[7] B. Steenhoek, M. M. Rahman, R. Jiles, and W. Le, "An empirical study of deep learning models for vulnerability detection," in *Proc. ICSE*, 2023, pp. 2237–2248.

[8] Y. Ganin and V. Lempitsky, "Unsupervised domain adaptation by backpropagation," in *ICML*, 2015, pp. 1180–1189.

[9] B. Sun and K. Saenko, "Deep CORAL: Correlation alignment for deep domain adaptation," in *ECCV Workshops*, 2016, pp. 443–450.

[10] M. Arjovsky, L. Bottou, I. Gulrajani, and D. Lopez-Paz, "Invariant risk minimization," *arXiv:1907.02893*, 2019.

[11] S. Sagawa, P. W. Koh, T. B. Hashimoto, and P. Liang, "Distributionally robust neural networks for group shifts," in *ICLR*, 2020.

[12] K. Muandet, D. Balduzzi, and B. Schölkopf, "Domain generalization via invariant feature representation," in *ICML*, 2013, pp. 10–18.

[13] C. Ni, L. Shen, X. Yang, Y. Zhu, and S. Wang, "MegaVul: A C/C++ vulnerability dataset with comprehensive code representations," in *Proc. MSR*. IEEE, 2024, pp. 738–742.

[14] Z. Feng, D. Guo, D. Tang, N. Duan, X. Feng, M. Gong, L. Shou, B. Qin, T. Liu, D. Jiang, and M. Zhou, "CodeBERT: A pre-trained model for programming and natural languages," in *Findings of EMNLP*, 2020, pp. 1536–1547.

[15] D. Guo, S. Ren, S. Lu, Z. Feng, D. Tang, S. Liu, L. Zhou, N. Duan, A. Svyatkovskiy, S. Fu et al., "GraphCodeBERT: Pre-training code representations with data flow," in *ICLR*, 2021.

[16] D. Guo, S. Lu, N. Duan, Y. Wang, M. Zhou, and J. Yin, "UniXcoder: Unified cross-modal pre-training for code representation," in *Proc. ACL*, 2022, pp. 7212–7225.

[17] Z. Gao, H. Wang, Y. Zhou, W. Zhu, and C. Zhang, "How far have we gone in vulnerability detection using large language models," *arXiv:2311.12420*, 2023.

[18] R. Safdar, D. Mateen, S. T. Ali, U. Ashfaq, and W. Hussain, "Data and context matter: Towards generalizing AI-based software vulnerability detection," *arXiv:2508.16625*, 2025.

[19] Y. Shi, J. Seely, P. Torr, N. Siddharth, A. Hannun, N. Usunier, and G. Synnaeve, "Gradient matching for domain generalization," in *ICLR*, 2022.

[20] K. Zhou, Z. Liu, Y. Qiao, T. Xiang, and C. C. Loy, "Domain generalization: A survey," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 45, no. 4, pp. 4396–4415, 2023.

[21] J. Wang, C. Lan, C. Liu, Y. Ouyang, T. Qin, W. Lu, Y. Chen, W. Zeng, and P. S. Yu, "Generalizing to unseen domains: A survey on domain generalization," *IEEE Trans. Knowl. Data Eng.*, vol. 35, no. 8, pp. 8052–8072, 2023.